Seguridad en Redes

Clase práctica: Ejercicios 6 y 9

Teoría de las Comunicaciones

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Certificados, TLS mutuo y una firma en cada pedido

Cómo vamos a usar las dos horas

Bloque	**Qué hacemos**
Repaso	Las herramientas que usan los dos ejercicios
Ejercicio 6	Proxy HTTPS: autenticar usuarios con certificados
Recreo	Diez minutos
Ejercicio 9	API sin sesiones: firmar cada pedido
Cierre	Comparación y ejercicios para practicar

La pregunta que guía la clase

¿Quién tiene que probar qué, y con qué clave?

1. ¿Qué piden?

Subrayar las propiedades: confidencialidad, integridad, autenticación, no repudio.

2. ¿Quién demuestra?

Quién se tiene que identificar y quién verifica. Del lado de quién se pide.

3. ¿Con qué clave?

El que demuestra usa su privada. El que verifica necesita la pública... y confiar en ella.

Agenda

1. Repaso
2. Ejercicio 6: un proxy que solo deja pasar a los nuestros
3. Ejercicio 9: una API que tiene que confiar
4. Cierre

Las cuatro propiedades que vamos a pedir

Confidencialidad

Solo el destinatario entiende el mensaje.

→ Cifrar (simétrica, o con la pública del receptor)

Integridad

Si alguien lo modifica en el camino, se detecta.

→ Hash protegido (firma)

Autenticación

Sé quién lo mandó.

→ firma, challenge-response

No repudio

Puedo probarle a un tercero quién lo mandó.

→ Firma digital

Las cuatro propiedades que vamos a pedir

Confidencialidad

Solo el destinatario entiende el mensaje.

→ Cifrar (simétrica, o con la pública del receptor)

Integridad

Si alguien lo modifica en el camino, se detecta.

→ Hash protegido (firma)

Autenticación

Sé quién lo mandó.

→ firma, challenge-response

No repudio

Puedo probarle a un tercero quién lo mandó.

→ Firma digital

No repudio ⇒ autenticación, pero no al revés. La disponibilidad no se logra con criptografía.

Simétrica vs. asimétrica

Simétrica

- Una sola clave K, compartida por los dos
- Rápida: AES para cifrar datos
- Problema: ¿cómo le hago llegar K al otro?
- Como los dos tienen K, no distingue cuál de los dos escribió

Asimétrica

- Un par por persona: K^+ pública y K^- privada
- Lenta: RSA, sobre datos chicos
- Resuelve la distribución: K^+ se publica
- Problema: ¿la K^+ que tengo es realmente de quien dice?

En la práctica se combinan: **asimétrica para autenticar y acordar una clave**, simétrica para los datos. Así trabaja TLS.

Una regla para no confundir las claves

Para que solo B pueda leer

Uso algo de B que todos tienen: su clave pública.

$$C = E_{K_B^+}(M)$$

Logra: confidencialidad

Una regla para no confundir las claves

Para que solo B pueda leer

Uso algo de B que todos tienen: su clave pública.

$$C = E_{K_B^+}(M)$$

Logra: confidencialidad

Para probar que fui yo (A)

Uso algo mío que solo yo tengo: mi clave privada.

$$F = E_{K_A^-}(H(M))$$

Logra: autenticación y no repudio

Notación de la guía: $E_Q(M)$ cifra M con la clave Q; $D_Q(C)$ descifra; $H(M)$ es el hash.

Hash: la huella digital del mensaje

- Entrada de cualquier tamaño, salida de tamaño fijo
- No inversible: de $H(M)$ no se recupera M
- Resistente a colisiones: no encuentro $M \neq M'$ con el mismo hash
- Un cambio mínimo cambia todo el resultado
- Hoy: SHA-256 (familia SHA-2) o SHA-3

```
H("fruit=Pera")
→ 9f2c...41ab

H("fruit=Pere")
→ 03d8...e7c2
```

Valores ilustrativos

Hash: la huella digital del mensaje

- Entrada de cualquier tamaño, salida de tamaño fijo
- No inversible: de $H(M)$ no se recupera M
- Resistente a colisiones: no encuentro $M \neq M'$ con el mismo hash
- Un cambio mínimo cambia todo el resultado
- Hoy: SHA-256 (familia SHA-2) o SHA-3

```
H("fruit=Pera")
→ 9f2c...41ab

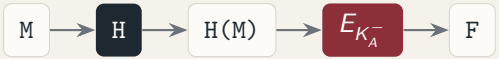
H("fruit=Pere")
→ 03d8...e7c2
```

Valores ilustrativos

Solo, no alcanza: el atacante cambia M y recalcula $H(M)$. Hay que proteger el hash con una clave.

Firma digital: el hash firmado con la privada

A firma



envía $M + F$

B verifica



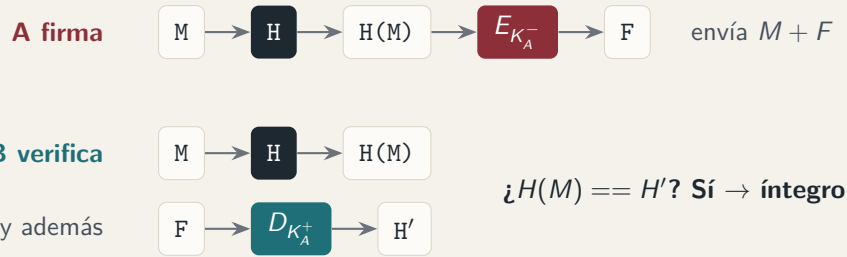
y además



¿$H(M) == H'$? **Sí** $\rightarrow$ **íntegro y de A**

Da **integridad, autenticación y no repudio**. No da confidencialidad: M viaja en claro.

Firma digital: el hash firmado con la privada



Da **integridad, autenticación y no repudio**. No da confidencialidad: M viaja en claro.

Pero B verifica con $K_A^+ \ldots$ ¿Cómo consiguió B la clave pública de A?¿Y si se la dio otra persona haciéndose pasar por A?

¿Esta clave pública es realmente de B?



1. B publica K_B^+; X la intercepta y le entrega a A su propia K_X^+.

¿Esta clave pública es realmente de B?



1. B publica K_B^+; X la intercepta y le entrega a A su propia K_X^+.
2. A cifra $E_{K_X^+}(M)$ creyendo que es para B.

¿Esta clave pública es realmente de B?



1. B publica K_B^+; X la intercepta y le entrega a A su propia K_X^+.
2. A cifra $E_{K_X^+}(M)$ creyendo que es para B.
3. X descifra, lee (o cambia) M, la vuelve a cifrar con K_B^+ y se la pasa a B. Nadie nota nada.

¿Esta clave pública es realmente de B?



1. B publica K_B^+; X la intercepta y le entrega a A su propia K_X^+.
2. A cifra $E_{K_X^+}(M)$ creyendo que es para B.
3. X descifra, lee (o cambia) M, la vuelve a cifrar con K_B^+ y se la pasa a B. Nadie nota nada.

Solución: un tercero en quien todos confían firma el vínculo identidad $\leftrightarrow$ clave. Eso es un certificado.

Certificado: la CA da fe de la clave

$$Cert_U = (\, U \,;\, K_U^+ \,;\, F \,) \quad \text{con} \quad F = E_{K_{CA}^-}(H(U + K_U^+))$$

Qué contiene (X.509)

- Sujeto: quién es U
- Su clave pública K_U^+
- Emisor: qué CA lo firmó
- Validez: desde / hasta
- Número de serie
- La firma F de la CA

Cómo lo verifica quien lo recibe

1. ¿$D_{K_{CA}^+}(F) == H(U + K_U^+)$?
2. ¿Está vigente y no fue revocado?
3. ¿De dónde sale K_{CA}^+? Ya estaba instalada: es la **raíz de confianza**.

TLS mutuo: el cliente también se presenta

Normalmente solo el servidor muestra certificado. Si el servidor lo pide, el handshake suma tres mensajes:

`CertificateRequest`

SERVIDOR → CLIENTE

"Mostrame tu certificado."

`Certificate`

CLIENTE → SERVIDOR

Manda $Cert_U$, firmado por la CA.

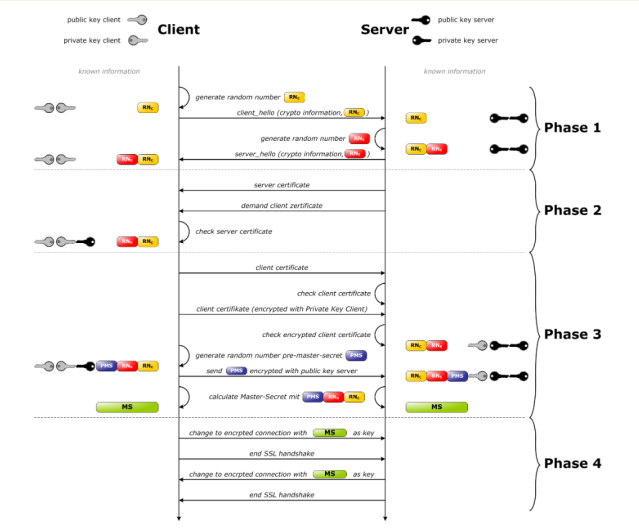
`CertificateVerify`

CLIENTE → SERVIDOR

Firma con K_U^- un resumen de todo el handshake, nonces incluidos.

En el diagrama SSL de la clase anterior: "client certificate encrypted with private key client".

El handshake SSL de la clase anterior



Los pasos del cliente ya estaban acá:

- **Fase 2:** *demand client certificate*
  → `CertificateRequest`
- **Fase 3:** *client certificate*
  → `Certificate`
- **Fase 3:** *client certificate encrypted with private key client*
  → `CertificateVerify`

El tercero es el que prueba que el cliente tiene K_C^-.

LA IDEA MÁS IMPORTANTE DE HOY

Mandar el certificado no prueba nada.

El certificado es público: cualquiera lo puede copiar. Lo que prueba identidad es firmar algo nuevo con la privada.

Ataque de repetición (replay)

El atacante no rompe nada: **graba un mensaje válido y lo reenvía tal cual**. La firma sigue siendo correcta, porque el mensaje no cambió.

Nonce

Valor aleatorio que se usa una sola vez. O lo elige el que verifica (un desafío), o recuerda los que ya vio.

Timestamp

Hora de creación. Se acepta solo dentro de una ventana corta, por ejemplo ± 5 minutos.

Clave: el nonce y el timestamp van **dentro de lo firmado**. Si no, el atacante los cambia.

Agenda

1. Repaso
2. Ejercicio 6: un proxy que solo deja pasar a los nuestros
3. Ejercicio 9: una API que tiene que confiar
4. Cierre

Ejercicio 6

DE PARCIAL

Una organización nos pide que implementemos una política de seguridad para su red. Dicha compañía desea que sólo usuarios autenticados puedan acceder a recursos Web en Internet conectándose mediante HTTPS con el Proxy. Explique cómo hacer para garantizar la autenticidad de los usuarios del lado del Proxy, aclarando dónde se instalarían los certificados digitales.



Ejercicio 6: leamos el enunciado con lupa

DE PARCIAL

Una organización nos pide que implementemos una política de seguridad para su red. Dicha compañía desea que sólo usuarios autenticados puedan acceder a recursos Web en Internet conectándose mediante HTTPS con el Proxy. Explique cómo hacer para garantizar la autenticidad de los usuarios del lado del Proxy, aclarando dónde se instalarían los certificados digitales.

Ejercicio 6: leamos el enunciado con lupa

DE PARCIAL

> *Una organización nos pide que implementemos una política de seguridad para su red. Dicha compañía desea que* ***sólo usuarios autenticados*** *puedan acceder a recursos Web en Internet conectándose mediante HTTPS con el Proxy. Explique cómo hacer para* ***garantizar la autenticidad de los usuarios*** *del lado del Proxy, aclarando dónde se instalarían los certificados digitales.*

- **Qué piden:** autenticación de los usuarios. No piden confidencialidad ni no repudio.

Ejercicio 6: leamos el enunciado con lupa

DE PARCIAL

> *Una organización nos pide que implementemos una política de seguridad para su red. Dicha compañía desea que* ***sólo usuarios autenticados*** *puedan acceder a recursos Web en Internet conectándose mediante HTTPS con el Proxy. Explique cómo hacer para* ***garantizar la autenticidad de los usuarios del lado del Proxy****, aclarando dónde se instalarían los certificados digitales.*

- **Qué piden:** autenticación de los usuarios. No piden confidencialidad ni no repudio.
- **Quién verifica:** el proxy. El que demuestra quién es, el usuario (con su K^-).

Ejercicio 6: leamos el enunciado con lupa

DE PARCIAL

> *Una organización nos pide que implementemos una política de seguridad para su red. Dicha compañía desea que* ***sólo usuarios autenticados*** *puedan acceder a recursos Web en Internet conectándose* ***mediante HTTPS con el Proxy****. Explique cómo hacer para* ***garantizar la autenticidad de los usuarios del lado del Proxy****, aclarando dónde se instalarían los certificados digitales.*

- **Qué piden:** autenticación de los usuarios. No piden confidencialidad ni no repudio.
- **Quién verifica:** el proxy. El que demuestra quién es, el usuario (con su K^-).
- **Qué nos exigen:** la conexión tiene que ser HTTPS, así que la autenticación va dentro de TLS.

Ejercicio 6: leamos el enunciado con lupa

DE PARCIAL

> *Una organización nos pide que implementemos una política de seguridad para su red. Dicha compañía desea que* ***sólo usuarios autenticados*** *puedan acceder a recursos Web en Internet conectándose* ***mediante HTTPS con el Proxy****. Explique cómo hacer para* ***garantizar la autenticidad de los usuarios del lado del Proxy****, aclarando* ***dónde se instalarían los certificados digitales****.*

- **Qué piden:** autenticación de los usuarios. No piden confidencialidad ni no repudio.
- **Quién verifica:** el proxy. El que demuestra quién es, el usuario (con su K^-).
- **Qué nos exigen:** la conexión tiene que ser HTTPS, así que la autenticación va dentro de TLS.
- **Qué hay que responder:** usa certificados, así que hay una CA. Decir dónde va cada uno.

Tres caminos posibles

Opción	**Cómo autentica**	**El problema**
Usuario y contraseña sobre el HTTPS	El proxy pide credenciales y las compara	Se roban, se adivinan, se reutilizan
Claves públicas cargadas en el proxy (como SSH)	Challenge-response con la privada del usuario	El proxy guarda N claves; altas y bajas a mano
Certificados de una CA + TLS mutuo	El proxy pide certificado en el handshake	**Elegida:** el proxy confía en una sola clave, K_{CA}^{+}

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	Qué se instala	Para qué

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^{+}\,;$ firma de la CA) y su K_P^{-}	

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^{+}\,;$ firma de la CA) y su K_P^{-}	Presentarse ante los usuarios

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^{+}\,;$ firma de la CA) y su K_P^{-}	Presentarse ante los usuarios
Proxy	Certificado de la CA (K_{CA}^{+})	

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^{+}\,;$ firma de la CA) y su K_P^{-}	Presentarse ante los usuarios
Proxy	Certificado de la CA (K_{CA}^{+})	Validar los certificados de los usuarios

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^+\,;$ firma de la CA) y su K_P^-	Presentarse ante los usuarios
Proxy	Certificado de la CA (K_{CA}^+)	Validar los certificados de los usuarios
Cada usuario U_i	$Cert_{U_i} = (U_i\,;\, K_{U_i}^+\,;$ firma de la CA) y su $K_{U_i}^-$	

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^{+}\,;$ firma de la CA) y su K_P^{-}	Presentarse ante los usuarios
Proxy	Certificado de la CA (K_{CA}^{+})	Validar los certificados de los usuarios
Cada usuario U_i	$Cert_{U_i} = (U_i\,;\, K_{U_i}^{+}\,;$ firma de la CA) y su $K_{U_i}^{-}$	Demostrar quién es en el handshake

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^+\,;$ firma de la CA) y su K_P^-	Presentarse ante los usuarios
Proxy	Certificado de la CA (K_{CA}^+)	Validar los certificados de los usuarios
Cada usuario U_i	$Cert_{U_i} = (U_i\,;\, K_{U_i}^+\,;$ firma de la CA) y su $K_{U_i}^-$	Demostrar quién es en el handshake
Cada usuario U_i	Certificado de la CA (K_{CA}^+)	

Solución: TLS mutuo con una CA propia

Paso 0: la organización arma su CA (o contrata una). La privada K_{CA}^{-} queda guardada y solo se usa para firmar certificados.

Dónde	**Qué se instala**	**Para qué**
Proxy	$Cert_P = (P\,;\, K_P^{+}\,;$ firma de la CA) y su K_P^{-}	Presentarse ante los usuarios
Proxy	Certificado de la CA (K_{CA}^{+})	Validar los certificados de los usuarios
Cada usuario U_i	$Cert_{U_i} = (U_i\,;\, K_{U_i}^{+}\,;$ firma de la CA) y su $K_{U_i}^{-}$	Demostrar quién es en el handshake
Cada usuario U_i	Certificado de la CA (K_{CA}^{+})	Validar el certificado del proxy

La conexión usuario → proxy, paso a paso



Qué chequea el proxy antes de dejar pasar

1. **¿Lo firmó nuestra CA?** — Certificado legítimo
   $D_{K^+_{CA}}(F) == H(U_i + K^+_{U_i})$
2. **¿Sigue vigente?** — No vencido ni dado de baja
   Fechas de validez y lista de revocados (CRL / OCSP)
3. **¿Tiene la privada?** — Es U_i, no una copia
   $D_{K^+_{U_i}}(\text{CertificateVerify}) == H(\text{handshake})$
4. **¿Puede navegar?** — Autorización
   U_i está en el grupo habilitado para usar el proxy

Si falla cualquiera: el proxy corta el handshake y no reenvía nada a Internet.

Ejercicio 6: respuesta modelo

PARA EL PARCIAL

1. La organización monta una CA. Su certificado (con K_{CA}^{+}) se instala en el proxy y en cada equipo de usuario.
2. El proxy genera (K_P^+, K_P^-), la CA le firma su certificado y se instala en el proxy.
3. Cada usuario U_i genera $(K_{U_i}^+, K_{U_i}^-)$ y obtiene $Cert_{U_i} = (U_i\,;\, K_{U_i}^+\,;\, E_{K_{CA}^-}(H(U_i + K_{U_i}^+)))$, que se instala en su equipo junto con $K_{U_i}^-$.
4. En el handshake TLS el proxy pide certificado al usuario, valida la firma de la CA y la vigencia, y verifica con $K_{U_i}^+$ la firma CertificateVerify. Si todo da bien, U_i queda autenticado; si no, se corta la conexión.

Recreo: 10 minutos

A la vuelta ej 9.

Agenda

1. Repaso
2. Ejercicio 6: un proxy que solo deja pasar a los nuestros
3. Ejercicio 9: una API que tiene que confiar
4. Cierre

Ejercicio 9 (de parcial)

La empresa Security First expone una API usando el protocolo HTTP. Dicha API tiene un método getPrice que devuelve el precio de frutas usando la siguiente url: `http://api.securityfirst.com/?method=getPrice&fruit=Pera`. *Es necesario agregar un método nuevo a la API que haga lo mismo que el getPrice pero que provea el servicio sólo a usuarios autenticados brindando integridad y no repudio del lado del servidor sobre los pedidos de los usuarios. Nota: es requisito no usar encripción sobre la totalidad del mensaje y no manejar sesiones.*

Ejercicio 9 (de parcial)

> *La empresa Security First expone una API usando el protocolo HTTP. Dicha API tiene un método getPrice que devuelve el precio de frutas usando la siguiente url:* `http://api.securityfirst.com/?method=getPrice&fruit=Pera`. *Es necesario agregar un método nuevo a la API que haga lo mismo que el getPrice pero que provea el servicio* ***sólo a usuarios autenticados*** *brindando* ***integridad y no repudio del lado del servidor*** *sobre los pedidos de los usuarios. Nota: es requisito no usar encripción sobre la totalidad del mensaje y no manejar sesiones.*

Qué piden: autenticación, integridad y no repudio. Demuestra el cliente; verifica el servidor.

Ejercicio 9 (de parcial)

> *La empresa Security First expone una API usando el protocolo HTTP. Dicha API tiene un método getPrice que devuelve el precio de frutas usando la siguiente url:* `http://api.securityfirst.com/?method=getPrice&fruit=Pera`. *Es necesario agregar un método nuevo a la API que haga lo mismo que el getPrice pero que provea el servicio* ***sólo a usuarios autenticados*** *brindando* ***integridad y no repudio del lado del servidor*** *sobre los pedidos de los usuarios. Nota: es requisito no usar* ***encripción sobre la totalidad del mensaje*** *y* ***no manejar sesiones****.*

Qué piden: autenticación, integridad y no repudio. Demuestra el cliente; verifica el servidor.

Qué no podemos usar: ni TLS ni cifrar la URL, ni un login con cookie. Cada pedido se defiende solo.

Ejercicio 9 (de parcial)

> *La empresa Security First expone una API usando el protocolo HTTP. Dicha API tiene un método getPrice que devuelve el precio de frutas usando la siguiente url:* `http://api.securityfirst.com/?method=getPrice&fruit=Pera`. *Es necesario agregar un método nuevo a la API que haga lo mismo que el getPrice pero que provea el servicio* ***sólo a usuarios autenticados*** *brindando* ***integridad y no repudio del lado del servidor*** *sobre los pedidos de los usuarios. Nota: es requisito no usar* ***encripción sobre la totalidad del mensaje*** *y* ***no manejar sesiones****.*

Qué piden: autenticación, integridad y no repudio. Demuestra el cliente; verifica el servidor.

Qué no podemos usar: ni TLS ni cifrar la URL, ni un login con cookie. Cada pedido se defiende solo.

Plan: arrancamos con la URL de hoy y la vamos mejorando. En cada versión: ¿qué hace el atacante? ¿cómo lo arreglamos?

Versión 0: la URL de hoy

EJERCICIO 9 · ITERACIÓN 0

```
http://api.securityfirst.com/?method=getPrice
&fruit=Pera
```

Cualquiera que conozca la URL obtiene el precio. El servidor no sabe quién pidió.

Versión 0: la URL de hoy

EJERCICIO 9 · ITERACIÓN 0

```
http://api.securityfirst.com/?method=getPrice
&fruit=Pera
```

Cualquiera que conozca la URL obtiene el precio. El servidor no sabe quién pidió.

¿Qué le falta?

Todo: el pedido no dice de quién es, nadie lo firma, nada impide modificarlo.

Autenticación ×	Integridad ×	No repudio ×	Anti-replay ×

Versión 0: la URL de hoy

EJERCICIO 9 · ITERACIÓN 0

```
http://api.securityfirst.com/?method=getPrice
&fruit=Pera
```

Cualquiera que conozca la URL obtiene el precio. El servidor no sabe quién pidió.

¿Qué le falta?

Todo: el pedido no dice de quién es, nadie lo firma, nada impide modificarlo.

Autenticación × Integridad × No repudio × Anti-replay ×

Primer arreglo: que el pedido diga quién es el cliente.

Versión 1: el cliente dice quién es

EJERCICIO 9 · ITERACIÓN 1

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33
```

El servidor tiene una lista de clientes habilitados y responde solo si el `clientId` está en la lista.

Versión 1: el cliente dice quién es

EJERCICIO 9 · ITERACIÓN 1

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33
```

El servidor tiene una lista de clientes habilitados y responde solo si el `clientId` está en la lista.

¿Qué hace el atacante?

Escribe `clientId=33` en su propia URL. El servidor no tiene forma de distinguirlo del cliente 33.

Autenticación × Integridad × No repudio × Anti-replay ×

Versión 1: el cliente dice quién es

EJERCICIO 9 · ITERACIÓN 1

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33
```

El servidor tiene una lista de clientes habilitados y responde solo si el `clientId` está en la lista.

¿Qué hace el atacante?

Escribe `clientId=33` en su propia URL. El servidor no tiene forma de distinguirlo del cliente 33.

Autenticación × Integridad × No repudio × Anti-replay ×

Lo que aprendimos: decir quién sos no es probarlo. Hace falta algo que solo tenga el cliente.

Versión 2: le agregamos una contraseña

EJERCICIO 9 · ITERACIÓN 2

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&pass=secreto
```

Cliente y servidor comparten un secreto. El servidor compara la contraseña recibida con la guardada.

Versión 2: le agregamos una contraseña

EJERCICIO 9 · ITERACIÓN 2

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&pass=secreto
```

Cliente y servidor comparten un secreto. El servidor compara la contraseña recibida con la guardada.

¿Qué hace el atacante?

La URL viaja en claro: **lee la contraseña** y la usa cuando quiera. También puede cambiar `Pera` por `Manzana`: nada lo detecta.

Autenticación × Integridad × No repudio × Anti-replay ×

Versión 2: le agregamos una contraseña

EJERCICIO 9 · ITERACIÓN 2

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&pass=secreto
```

Cliente y servidor comparten un secreto. El servidor compara la contraseña recibida con la guardada.

¿Qué hace el atacante?

La URL viaja en claro: **lee la contraseña** y la usa cuando quiera. También puede cambiar `Pera` por `Manzana`: nada lo detecta.

Autenticación × Integridad × No repudio × Anti-replay ×

Lo que aprendimos: el secreto no puede viajar (y no podemos cifrar la URL ni hacer login con cookie). Hay que mandar algo *calculado* con el secreto y con el pedido.

Versión 3: un hash con clave (MAC)

EJERCICIO 9 · ITERACIÓN 3

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&mac=HMAC_K(method, fruit, clientId)
```

La clave K la comparten cliente y servidor, pero ya no viaja. El servidor recalcula el MAC y compara.

Versión 3: un hash con clave (MAC)

EJERCICIO 9 · ITERACIÓN 3

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&mac=HMAC_K(method, fruit, clientId)
```

La clave K la comparten cliente y servidor, pero ya no viaja. El servidor recalcula el MAC y compara.

Lo que ahora sí funciona

Si cambia la fruta, el MAC no coincide. Sin K no puede fabricar uno válido.

El problema

El servidor **también** tiene K. El cliente puede decir: "ese pedido lo fabricó el servidor".

Autenticación ✓	Integridad ✓	No repudio ×	Anti-replay ×

Versión 3: un hash con clave (MAC)

EJERCICIO 9 · ITERACIÓN 3

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&mac=HMAC_K(method, fruit, clientId)
```

La clave K la comparten cliente y servidor, pero ya no viaja. El servidor recalcula el MAC y compara.

Lo que ahora sí funciona

Si cambia la fruta, el MAC no coincide. Sin K no puede fabricar uno válido.

El problema

El servidor **también** tiene K. El cliente puede decir: "ese pedido lo fabricó el servidor".

Autenticación ✓ Integridad ✓ No repudio × Anti-replay ×

Lo que aprendimos: para no repudio, la prueba la tiene que poder generar **uno solo**: la clave privada del cliente.

Versión 4: firma digital

EJERCICIO 9 · ITERACIÓN 4

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&sig=FIRMA
```

$$FIRMA = E_{K^-_{C_{33}}}(H(\text{method, fruit, clientId}))$$

El servidor guarda $K^+_{C_{33}}$ y verifica: $D_{K^+_{C_{33}}}(FIRMA) == H(\ldots)$. (Es la resolución del handout.)

Versión 4: firma digital

EJERCICIO 9 · ITERACIÓN 4

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&sig=FIRMA
```

$$FIRMA = E_{K^-_{C_{33}}}(H(\text{method, fruit, clientId}))$$

El servidor guarda $K^+_{C_{33}}$ y verifica: $D_{K^+_{C_{33}}}(FIRMA) == H(\ldots)$. (Es la resolución del handout.)

¿Qué hace el atacante?

Copia la URL entera y la **reenvía tal cual**, mil veces. La firma sigue siendo válida, y el cliente puede decir "yo la mandé una sola vez".

Autenticación ✓ Integridad ✓ No repudio ✓ Anti-replay ×

Versión 4: firma digital

EJERCICIO 9 · ITERACIÓN 4

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&sig=FIRMA
```

$$FIRMA = E_{K^-_{C_{33}}}(H(\text{method, fruit, clientId}))$$

El servidor guarda $K^+_{C_{33}}$ y verifica: $D_{K^+_{C_{33}}}(FIRMA) == H(\ldots)$. (Es la resolución del handout.)

¿Qué hace el atacante?

Copia la URL entera y la **reenvía tal cual**, mil veces. La firma sigue siendo válida, y el cliente puede decir "yo la mandé una sola vez".

Autenticación ✓ Integridad ✓ No repudio ✓ Anti-replay ×

Lo que aprendimos: cada pedido tiene que ser único, y eso tiene que estar **dentro** de lo firmado.

Versión 5: timestamp y nonce

EJERCICIO 9 · ITERACIÓN 5

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&ts=1792500000&nonce=8f3a91c2&sig=FIRMA
```

$$FIRMA = E_{K_{C_{33}}^{-}}(H(\text{method, fruit, clientId, ts, nonce}))$$

- **ts:** el servidor rechaza pedidos fuera de ± 5 minutos.
- **nonce:** el servidor recuerda los nonces recientes y rechaza los repetidos.

Versión 5: timestamp y nonce

EJERCICIO 9 · ITERACIÓN 5

```
http://api.securityfirst.com/?method=getPriceAuth
&fruit=Pera&clientId=33&ts=1792500000&nonce=8f3a91c2&sig=FIRMA
```

$$FIRMA = E_{K_{C_{33}}^{-}}(H(\text{method, fruit, clientId, ts, nonce}))$$

- **ts:** el servidor rechaza pedidos fuera de ± 5 minutos.
- **nonce:** el servidor recuerda los nonces recientes y rechaza los repetidos.

Autenticación ✓ Integridad ✓ No repudio ✓ Anti-replay ✓

Lo que construimos

	Agregamos	Aut.	Integ.	No rep.	Replay
V0	nada	×	×	×	×
V1	`clientId`	×	×	×	×
V2	contraseña	×	×	×	×
V3	MAC con clave compartida	✓	✓	×	×
V4	firma con la privada	✓	✓	✓	×
V5	`ts` + `nonce` firmados	✓	✓	✓	✓

Ahora sí podemos responder los tres incisos: salen de lo que fuimos construyendo.

a.1 · Qué tiene que tener cada parte

Cada cliente C_i

- Su par $(K_{C_i}^{+}, K_{C_i}^{-})$; la privada no sale de su máquina
- Su identificador `clientId`
- Los algoritmos acordados, por ejemplo RSA + SHA-256

El servidor

- Una tabla `clientId` $\rightarrow K_{C_i}^{+}$, cargada antes por un canal seguro
- Los mismos algoritmos
- Reloj sincronizado y los nonces recientes

El servidor no necesita una privada propia: solo verifica.

a.3 · Qué hace el cliente

1. **Arma la cadena canónica**
   Todos los parámetros menos `sig`, en un orden fijo acordado
2. **Calcula el hash**
   $D = H(\text{cadena})$
3. **Firma con su privada**
   $FIRMA = E_{K_C^-}(D)$
4. **Agrega `&sig=` y manda el GET**
   La firma va en Base64URL; el resto de la URL, en claro

Todo lo que el servidor usa para decidir tiene que estar dentro de lo firmado.

a.3 · Qué hace el servidor

1. **Busca K_C^+ por `clientId`** — Cliente conocido
   Si no existe, rechaza el pedido
2. **Chequea `ts` y `nonce`** — Anti-replay
   `ts` dentro de ± 5 min y nonce nunca visto
3. **Recalcula el hash** — Integridad
   $D' = H(\text{cadena canónica recibida})$
4. **Verifica la firma** — Autenticación
   ¿$D_{K_C^+}(FIRMA) == D'$?
5. **Responde y guarda pedido + FIRMA** — No repudio
   El registro firmado es la evidencia ante un tercero

Ejercicio 9: respuesta modelo

PARA EL PARCIAL

a.1 Cada cliente C_i tiene $(K^+_{C_i}, K^-_{C_i})$ y un `clientId`. El servidor guarda `clientId` $\rightarrow K^+_{C_i}$, intercambiadas antes de forma segura. Acuerdan RSA + SHA-256.

a.2 `http://api.securityfirst.com/?method=getPriceAuth &fruit=Pera&clientId=33&ts=...&nonce=...&sig=FIRMA`

a.3 El cliente calcula $FIRMA = E_{K^-_{C_i}}(H(\text{method, fruit, clientId, ts, nonce}))$. El servidor busca $K^+_{C_i}$, chequea `ts` y nonce, verifica $D_{K^+_{C_i}}(FIRMA) == H(\ldots)$ y guarda el pedido firmado.

Autenticación, integridad y no repudio, sin cifrar el mensaje y sin sesiones.

Agenda

1. Repaso
2. Ejercicio 6: un proxy que solo deja pasar a los nuestros
3. Ejercicio 9: una API que tiene que confiar
4. Cierre

Dos ejercicios, una misma idea

	Ejercicio 6	**Ejercicio 9**
Piden	Autenticación	Autenticación, integridad, no repudio
Contexto	Conexión HTTPS (con sesión)	Pedidos sueltos (sin sesión)
Mecanismo	TLS mutuo con certificados	Firma digital en cada URL
El verificador guarda	Solo K_{CA}^{+}	K^{+} de cada cliente
Prueba de la privada	CertificateVerify, una vez	La firma, en cada pedido
Anti-replay	Nonces del handshake	`ts` + nonce en la URL

En los dos: **probar que tenés la privada, firmando algo nuevo**. Cambia cuándo y cuántas veces.

Para practicar: ejercicios de la guía

Ejercicio 7 · El WiFi del hotel

Autenticidad del proxy y confidencialidad de los huéspedes.

Pista: es el Ej. 6 al revés. ¿Quién se autentica ahora? ¿Dónde va el certificado de la CA?

Ejercicio 8 · Peleas internas

No repudio en todos los mensajes entre empleados.

Pista: es el Ej. 9 aplicado a mensajes. ¿Por qué no alcanza un MAC con una clave compartida?

¿Preguntas?